IEMS 469 Reinforcement Learning, Assignment 2: Deep Q-Network

Özgün Barış Kır, student number 3676663, GitHub username ozgunbaris

November 5, 2026

Repository and how to run the code

Özgün Barış Kır (3676663). GitHub username: **ozgunbaris**. Repository: **mlds-rl-assignment2**, https://github.com/ozgunbaris/mlds-rl-assignment2.

All code is in the repository and `README.md` lists every command. Short version (Python 3.12, run from the repository root):

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m pytest tests -q                                    # 11 mechanical checks
python -m dqn.cartpole_dqn --out results/cartpole/main_seed0 --seed 0 \
    --episodes 1000 --eps-final 0.1 --no-stop-when-solved \
    --eval-periodic-episodes 20                              # (a), one seed
scripts/run_cartpole.sh                                      # (a), all 23 runs
python -m dqn.make_plots_cartpole                            # (a) figures
python -m dqn.validate_cartpole                              # (a) validation
scripts/run_mspacman.sh                                      # (b) seed 0 + 500 games
scripts/run_mspacman_seed1.sh                                # (b) seed 1 + 500 games
python -m dqn.make_plots_mspacman --runs results/mspacman/main_seed0
python -m dqn.validate_mspacman --run results/mspacman/main_seed0
```

Every `results/*` folder holds the logs (`episodes.csv`, `heldout_q.csv`, `eval_periodic.csv`), the network weights, the roll-out rewards (`eval_rewards_*.npy`, `eval_summary_*.json`), the figures, and `validation.md` with the numbers quoted below.

Compute. All runs were made on a cloud machine with 4 CPU cores and no GPU. The 23 CartPole runs ran one after another on one core while MsPacman trained on the other three: seed 0 on 3 threads, and seed 1 on 1 thread once the CartPole runs had finished. The cloud machine restarted during training and stopped both MsPacman runs; the networks reported here are their last checkpoints, after 1,100 games (0.92 hours of training) for seed 0 and 300 games (0.25 hours) for seed 1. Training was then resumed from these checkpoints (`scripts/run_mspacman_continue.sh`, results in `results/mspacman/cont_seed*`); `HANDOFF.md` describes how to fold the longer runs into this report. The handout asks for a single GPU on the deepdish cluster; the cluster was not reachable from this machine, and `scripts/nomad/make_job.sh` generates the job that continues the MsPacman training there.

The DQN implementation (shared by both games)

`dqn/core.py`, `dqn/replay.py` and `dqn/networks.py` are shared. One training step does the following.

1. Act ϵ-greedily on the online network Q_θ: a uniformly random action with probability ϵ, otherwise $\arg\max_a Q_\theta(s, a)$. The exploration rate follows a piecewise-linear schedule (an ϵ scheduler).
2. Store the transition (s, a, r, s', d) in the **replay buffer**. d is the bootstrap mask: it is 1 only when the episode really ended (pole fell, cart left the track, game over, or a lost life in MsPacman). A time-limit truncation is not terminal: s' is a real state and its value is bootstrapped (edge cases below).
3. Sample a uniform minibatch from the buffer and form the **Double DQN** target with the **target network** $Q_{\bar\theta}$:
$$y = r + \gamma\,(1 - d)\,Q_{\bar\theta}(s', \arg\max_{a'} Q_\theta(s', a')).$$
The online network chooses the next action and the target network evaluates it, which removes most of the upward bias of $\max_{a'} Q_{\bar\theta}(s', a')$ (van Hasselt et al. 2016). `--no-double` gives the original DQN target.
4. Take one Adam step on the **Huber loss** between $Q_\theta(s, a)$ and y (y carries no gradient), with the gradient norm clipped at 10. The learning rate decays linearly (a learning-rate scheduler).
5. Copy θ into $\bar\theta$ every fixed number of updates.

The three required components are therefore a neural network (Q_θ), a replay buffer (`VectorReplay`, `FrameStackReplay`) and a target network ($Q_{\bar\theta}$). Ablations that remove each of them are reported in part (a).

Maximum Q-value plots. For plot (i) every training episode logs, from the forward passes the agent already makes to act, the mean over the states it visited of $\max_a Q_\theta(s, a)$, the largest such value in the episode and the value of the first state. Each run also tracks the average of $\max_a Q_\theta$ over a fixed set of held-out states collected with a random policy before training (Mnih et al. 2015, Fig. 2), which does not move with the agent's own state distribution.

Part (a): CartPole-v1

Setup

The Q-network is a 4-128-128-2 MLP with ReLU. Its inputs are divided by $(2.4, 3, 0.21, 3.5)$, the cart-position and pole-angle failure limits and typical velocities, so that all four are of order 1. Discount factor $\gamma = 0.95$ as prescribed. Adam with learning rate 5×10^{-4} decaying linearly to 5×10^{-5} over the 1000 training episodes; minibatch 64; replay buffer of 50,000 transitions; updates start after 1000 steps; one update per environment step; the target network is refreshed every 500 updates. ϵ decays linearly from 1 to 0.1 between steps 1,000 and 21,000 and stays at 0.1.

Every 25 training episodes the greedy policy plays 20 evaluation episodes on a separate environment instance. The network with the best such score (ties go to the later network) is kept as the trained model; the reasons for this choice and the behaviour of the network after the last episode are part of the edge cases. After training, 500 new greedy episodes (new seeds) are rolled out with the kept network and with the final network. Five seeds were run.

Results

The task is solved. The kept networks of three of the five seeds balance the pole for all 500 steps in every one of the 500 roll-out episodes (mean 500.0, standard deviation 0.0); the other two reach 500 steps in 97.8 % and 85.2 % of the episodes. In the five main runs every roll-out failure, of the kept and of the final networks, is the cart leaving the track; the pole never fell.

seed	kept network: training episode (epsilon)	its 500 greedy roll-outs: mean (std)	share at 500 steps	network after the last episode: mean	training MA100 at the end
0	950 (0.10)	500.0 (0.0)	100 %	173.4	125.3
1	550 (0.31)	495.4 (33.6)	97.8 %	98.0	85.7
2	275 (0.71)	455.8 (106.0)	85.2 %	88.6	89.5
3	800 (0.10)	500.0 (0.0)	100 %	103.9	88.3
4	175 (0.86)	500.0 (0.0)	100 %	98.3	101.4

Mean over the five seeds of the kept networks' roll-out means: 490.2. One run (1000 episodes, 53,000 to 68,000 steps) takes 83 to 114 seconds on one CPU core. The figures below are those of seed 0, whose network was kept late in training (episode 950), so that its Q-values are also converged; the corresponding figures of all seeds are in `results/cartpole/figures/per_run/`.

(i) Maximum Q-values. The value of the visited states rises from 0 to the bound $1/(1-\gamma) = 20$ in about 600 episodes and stays there (mean of the last 100 episodes 19.72 for seed 0, 19.72 to 20.37 over the five seeds). Every step pays +1 and the time limit is bootstrapped, so a policy that never fails has $Q = 20$ exactly, which is where the learned values settle; the largest value of an episode is slightly above it (20.6 to 21.1 over the last 100 episodes), which is the overestimation measured in test C3. The average over the 500 fixed held-out states (taken from a random policy, so they include falling poles) settles lower, at 18.1. The held-out curve rises in steps: each step is a copy of the online network into the target network (every 500 updates), the moment when the regression target moves.

(ii) Rewards. The training reward is collected with ϵ-greedy actions, and after step 21,000 one action in ten is random. In states where the margin is small one random push ends the episode (the trace in the tests shows a state where the wrong action is worth 6.3 against 20.7, i.e. failure within about 7 steps), so the 100-episode moving average stays near 100 even when the greedy policy is perfect. The green markers in the same figure are the greedy checks (20 separate evaluation episodes every 25 training episodes): they reach 500 already at episode 125 (seed 0; 125 to 225 for four seeds), when ϵ is still about 0.9, and fluctuate between about 100 and 500 once ϵ is at its floor. This instability is analysed in the edge cases.

(iii) Roll-outs. The kept network of seed 0 scores 500 in all 500 greedy episodes.

Edge cases (part a)

- **The 500-step time limit is a truncation, not a terminal state.** CartPole-v1 returns `truncated = True` after 500 steps. The state (cart position and velocity, pole angle and angular velocity) carries no clock, so the value of a state cannot depend on how many steps happen to remain. The bootstrap mask is therefore \$d = \$ `terminated` only, and the target at a truncated step is $1 + \gamma \max Q(s')$, not 1. Treating the limit as terminal gives some visits of a state the target 1 and the others about 20. Test C1 measures the resulting bias on a problem where it can be computed exactly, and the `truncterm` runs measure its effect on CartPole (tests below).
- **The reward of the last step and the bound on Q.** CartPole pays +1 on every step, including the step on which the pole falls, so the target of a terminal transition is exactly 1 and every Q lies in $[1, 1/(1-\gamma)] = [1, 20]$. The learned values respect this on the states the agent visits (19.72 at the end of seed 0); predictions above 20 are overestimation and are measured in test C3.
- **A discount factor of 0.95 makes the agent short-sighted, and the cart drifts off the track.** A failure T steps ahead lowers the value by $20 \cdot 0.95^T$: by 7.2 at $T = 20$, 0.92 at $T = 60$

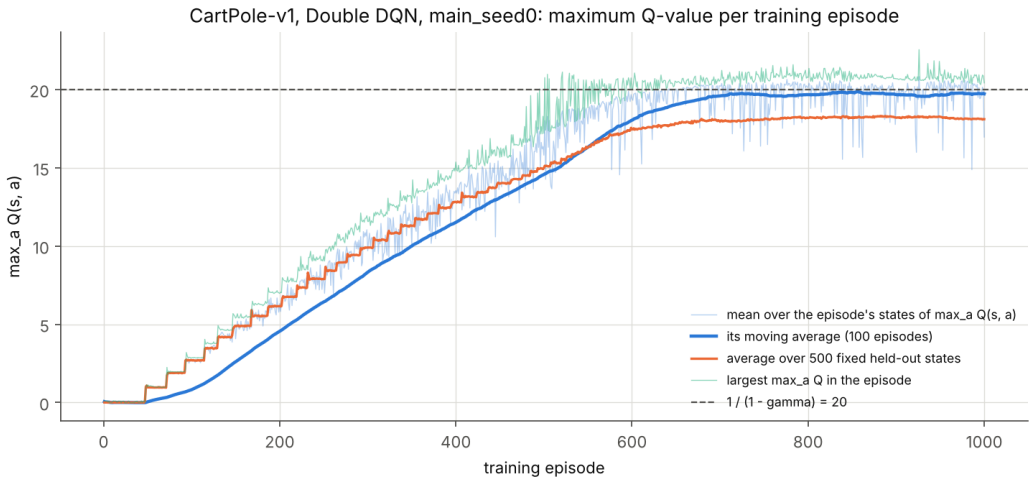


Figure 1: (a)(i) CartPole-v1, seed 0: maximum Q-value per training episode. The dashed line is $1/(1-\gamma) = 20$, the exact value of a policy that never fails.

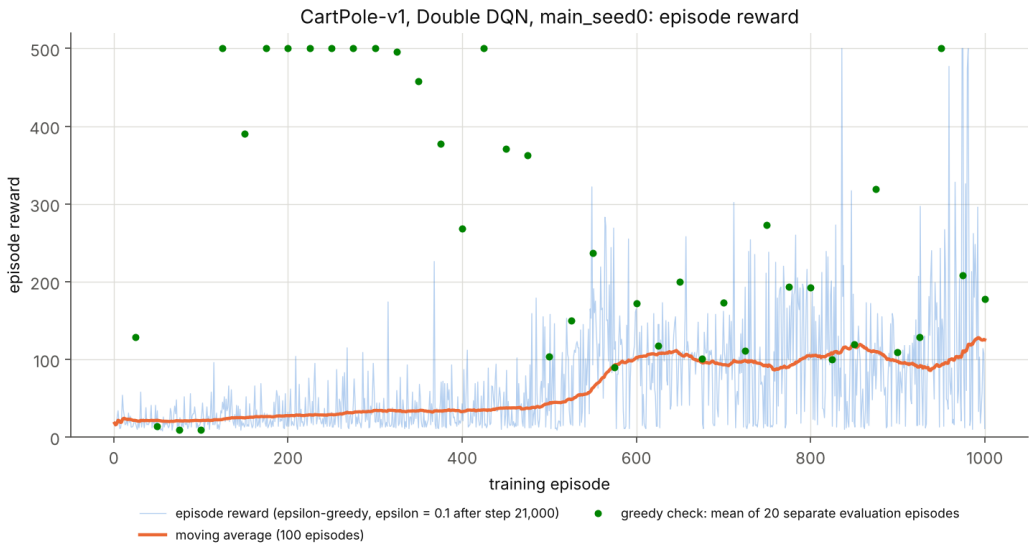


Figure 2: (a)(ii) CartPole-v1, seed 0: episode reward (with ϵ-greedy exploration) and its 100-episode moving average.

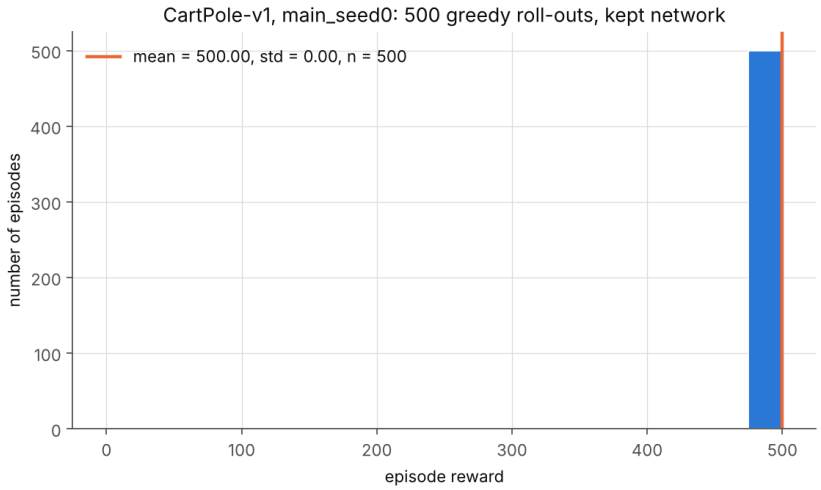


Figure 3: (a)(iii) CartPole-v1, seed 0: 500 greedy roll-out episodes of the trained (kept) network.

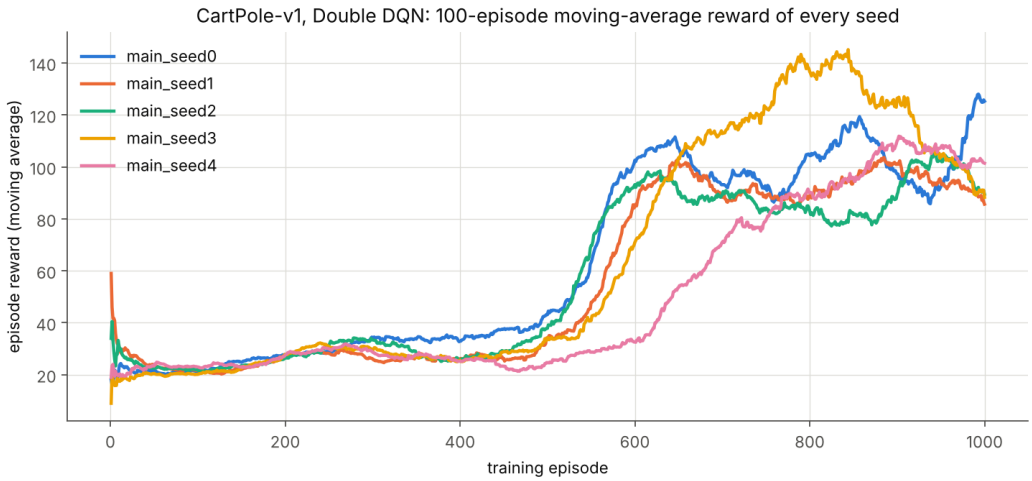


Figure 4: CartPole-v1: 100-episode moving-average reward of all five seeds.

and 0.12 at $T = 100$. A slow drift of the cart toward the edge is therefore almost invisible to the value until it can no longer be corrected. The first trial network (600 episodes, ϵ floor 0.02, unscaled inputs; `results/cartpole/preliminary/cp_try0`) balanced the pole but lost the cart in all 500 roll-outs, after 115 steps on average, and in the five main runs every roll-out failure is a cart-position failure. The cart-position map of the kept network (Figure 6) shows the cause: with the pole upright, the value falls to about 18 only at the left edge while moving left; the right edge is barely represented, and in states the agent never visits (fast motion near an edge) the network extrapolates up to 23.3, above the bound. The configuration whose kept networks hold the cart on the track for 500 steps combines inputs scaled to order 1, longer training, more exploration (next item) and model selection; the preliminary runs did not isolate the contribution of each.

- **Instability once the task is learned, and how the trained network is chosen.** Once the agent balances, almost every target is close to 20 and the two actions differ by very little: the median absolute action gap $|Q(s, \text{right}) - Q(s, \text{left})|$ on visited states is 0.044 (test C5). A small change of the weights then flips the greedy action in many states, so the greedy policy keeps changing even when the loss is small. The greedy checks show it (Figure 2): repeatedly perfect between episodes 125 and 425, then between about 100 and 500 once ϵ reached its floor (episode 558). All five networks after the last episode score 88.6 to 173.4. The preliminary runs (`results/cartpole/preliminary/README.md`) show the same with every variation tried (larger buffer and batch, soft target updates, a wider network, a stopping rule). The remedy used here is model selection on separate data: the network with the best greedy check (20 evaluation episodes on a different environment instance) is kept, and it is judged on 500 new episodes. Both networks are reported.
- **Exploration keeps the data informative.** The perfect greedy policies appear early, when ϵ is still large and the replay buffer is full of near-failures: in the preliminary runs with a stopping rule active from the start, four of four seeds stopped at episodes 125 to 250 (ϵ 0.75 to 0.91) with Q-values of only 3.4 to 8.9 and scored 500 in all 500 roll-outs. The greedy policy only needs the sign of the action gap, not the value. With an ϵ floor of 0.02 the stopped networks were worse (97.4 % to 99.0 % at 500 steps, one seed never stopped); with a floor of 0.1 the best-check networks of four of four seeds were perfect. The final configuration therefore keeps $\epsilon = 0.1$.
- **Warm-up of the replay buffer.** No update is made before 1000 transitions are stored, so the first minibatches are not dominated by a handful of correlated transitions; the buffer of 50,000 transitions holds the last 100 to about 2,500 episodes, depending on their length.
- **Seeds.** DQN on CartPole varies a lot between seeds (kept networks 455.8 to 500.0), so every configuration was run with 5 seeds (main) or 2 to 3 seeds (ablations), and all of them are reported.

Tests that validate correctness (part a)

The 11 checks in `tests/test_dqn.py` cover the mechanics (the frame-stack replay buffer reproduces every stored transition across episode boundaries and wrap-around, the Bellman target masks terminal steps, Double DQN evaluates the online argmax with the target network, the target network stays frozen between copies, rewards are clipped only inside the target, the preprocessing matches a signed computation) and are not repeated here. The experiments below test whether the *algorithm* behaves as reinforcement learning predicts. `python -m dqn.validate_cartpole` runs them (test C7 summarises the ablation runs of `scripts/run_cartpole.sh`) and writes `results/cartpole/validation/validation.md`.

1. **C1: a deterministic problem with known answers (Figure 7).** `dqn/toy_chain.py` is an 8-state chain, observed as one-hot vectors, with deterministic left/right moves; moving into the last state pays 1 and ends the episode. Its exact Q^* follows from value iteration. The same network class, replay buffer, Double DQN update and target network as for CartPole were trained for 20,000 steps from a uniformly random behaviour policy. The largest error $\max_{s,a} |Q - Q^*|$ was 0.0017, 0.0000 and 0.0000 for Double DQN (3 seeds), 0.0044, 0.0000, 0.0000 for the DQN target, and 0.0217, 0.0046,

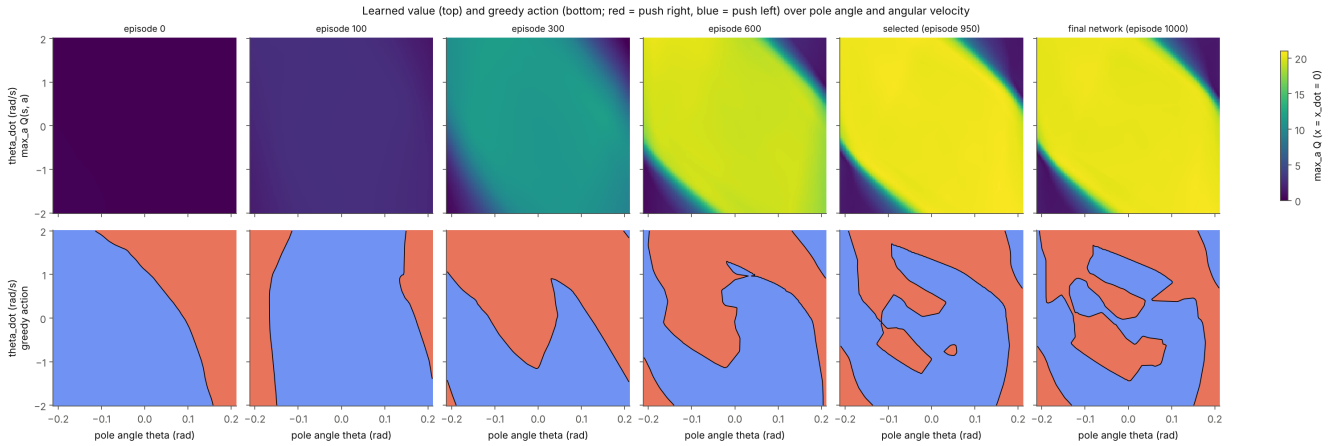


Figure 5: Non-standard plot: learned value (top) and greedy action (bottom) of seed 0 over pole angle and angular velocity (cart centred and at rest), from the untrained network to the kept and the final network.

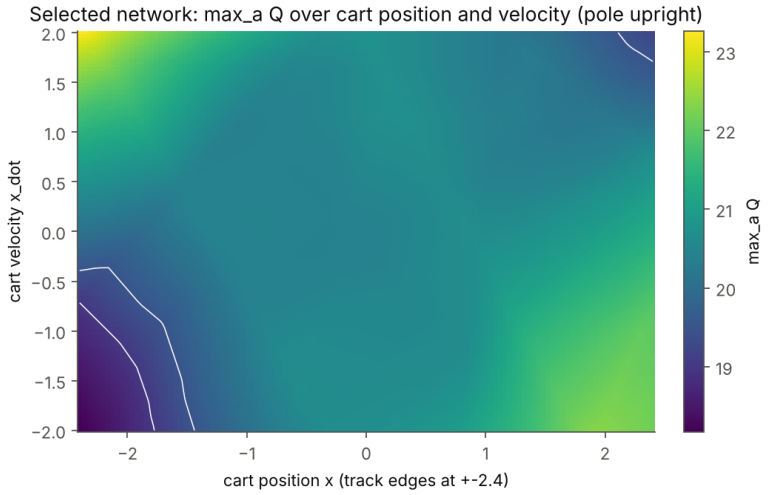


Figure 6: Kept network of seed 0: maximum Q-value over cart position and velocity with the pole upright.

0.0093 without a target network; the greedy policy was optimal in all 9 runs.

2. **C1: the time-limit edge case, with predicted numbers.** In a second chain every step pays 1, nothing terminates, and episodes are cut after 50 steps. Bootstrapping at the cut must give $Q = 1/(1-\gamma) = 20$ everywhere: measured 20.02, 20.01 and 19.99. If the cut is wrongly treated as terminal, a fraction $p = 1/50$ of the targets is 1 instead of $1+\gamma Q$. With a squared loss the fixed point is the mean target, $Q = 1/(1-\gamma(1-p)) = 14.49$: measured 15.28, 15.85 and 15.22 with the gradient-norm clipping at 10 that every run uses, and 14.96, 15.22 and 14.81 without it. Clipping explains part of the gap, because it shrinks exactly the minibatches that contain the large errors of the wrong targets; the remaining 0.5 (3 %) is not explained by this simple model, which assumes that every state-action pair sees the same share p of wrong targets. With the Huber loss the wrong targets lie far below Q, so each contributes a gradient of fixed size 1 instead of one proportional to its error; the fixed point solves $(1-p)(1+\gamma Q-Q) = p$, i.e. $Q = (1-p/(1-p))/(1-\gamma) = 19.59$: measured 19.43, 19.87, 19.66. The squared-loss prediction (14.49) was made before any measurement; the Huber prediction was derived after the first measurement (19.65 on average) disagreed with it, and the squared-loss runs were then made to check the explanation. The Huber loss therefore makes the implementation robust to rare wrong targets, which is a second reason to use it besides the bounded gradient.
3. **The time limit in CartPole itself.** With $\epsilon = 0.1$ the ϵ-greedy training episodes rarely survive 500 steps: the limit was reached in 4, 0, 0, 1 and 0 of the 1000 training episodes of the five seeds. Until the first truncation the `truncterm` runs (time limit treated as terminal) are the same computation as the main runs; seed 1 is identical to the last digit (kept network 495.4, final 98.0), and seed 0 departs only after episode 836 (kept 500.0 in both; final network 470.6 against 173.4, a divergence of two chaotic training runs rather than an effect of the time limit, since only 4 truncated transitions entered the buffer). On CartPole with this configuration the edge case exists in principle but almost never occurs, and C1 is the test that isolates it.
4. **C2: the scale of Q follows the discount factor and the reward.** A policy that never fails has $Q = r/(1-\gamma)$ exactly, so the learned value must scale with the discount factor and with the reward (Figure 8). Over the last 100 of 1000 episodes the visited-state value averages 9.78 and 9.68 with $\gamma = 0.9$ (exact value 10; 2 seeds), 19.72 to 20.37 with $\gamma = 0.95$ (exact 20; 5 seeds), and 39.74 with every reward doubled (exact 40). With $\gamma = 0.99$ it reaches 73.5 and 76.2 and is still rising (exact 100). This is the expected speed: each copy into the target network acts roughly like one Bellman backup, and a sequence of k backups from 0 reaches $100(1-0.99^k)$, which is 70.7 and 66.2 after the 122 and 108 copies of these runs, against 20.0 of 20 for $\gamma = 0.95$ after 134 copies. The measured values are of that size and somewhat higher, plausibly because the online network keeps fitting between copies, so a period does a little more than one backup. The kept networks score 500.0 and 500.0 ($\gamma = 0.9$), 500.0 and 496.3 ($\gamma = 0.99$) and 500.0 (doubled reward).
5. **C3: overestimation against the true return.** The kept network of seed 0 played 20 greedy episodes with the time limit raised to 2000 steps; all 20 lasted the full 2000 steps, four times the training limit, so the true discounted return of every state in the first 500 steps is 20.00 (the tail beyond 2000 steps is below 10^{-30}). The predicted $Q(s_t, a_t)$ averaged 20.67: Double DQN still overestimates, by 0.67 (3.4 %), and by at most 1.05 in any state.
6. **C4: the first 10 steps of a greedy episode.** The table follows seed 0's kept network. While the pole is nearly upright the two actions are almost equally good (20.43 against 20.41 at $t = 0$), and the Bellman error $y - Q(s,a)$ is below 0.04 in absolute value in the first five steps (at most 0.22 in the ten). At $t = 6$ the pole leans right ($\theta = 0.103$) and rotates right ($\dot\theta = 1.23$): pushing left is worth 6.32, which is the value of failing in about 7 steps ($\sum_{k<7} 0.95^k = 6.0$), and pushing right 20.74, so the agent pushes right and keeps pushing right until the rotation slows (Q(left) recovers to 8.70, 12.64, 17.64).

t	x	x_dot	theta	theta_dot	Q(left)	Q(right)	action	r	y = r + g max Q(s')	y - Q(s,a)
0	+0.013	+0.040	+0.028	-0.027	20.434	20.406	left	1	20.432	-0.0022
1	+0.013	-0.156	+0.027	+0.274	20.455	20.453	left	1	20.441	-0.0135
2	+0.010	-0.351	+0.032	+0.575	20.465	20.420	left	1	20.427	-0.0380
3	+0.003	-0.547	+0.044	+0.878	20.449	20.410	left	1	20.423	-0.0265
4	-0.008	-0.743	+0.062	+1.184	19.906	20.445	right	1	20.454	+0.0087
5	-0.023	-0.548	+0.085	+0.911	20.477	20.412	left	1	20.698	+0.2210
6	-0.034	-0.744	+0.103	+1.229	6.321	20.735	right	1	20.631	-0.1039
7	-0.048	-0.551	+0.128	+0.971	8.700	20.665	right	1	20.568	-0.0968
8	-0.059	-0.358	+0.147	+0.721	12.637	20.598	right	1	20.578	-0.0194
9	-0.067	-0.165	+0.162	+0.478	17.644	20.609	right	1	20.563	-0.0459

7. **C5: mirror symmetry.** The dynamics are unchanged when every state variable changes sign and left and right are swapped, so a correct Q-function satisfies $Q(s, \text{left}) = Q(-s, \text{right})$. On 5000 states visited by the kept network the mean absolute difference is 0.29 and the action gaps of s and $-s$ have correlation 0.970. The greedy actions agree in 66 % of the states, but this is concentrated where the actions are interchangeable: the agreement is 81.9 % where the gap exceeds 0.1 (32 % of the states), 88.6 % where it exceeds 0.5 and 95.4 % where it exceeds 1 (14 % of the states); the median gap is 0.044. The policy is symmetric where the choice matters.

8. **C7: removing each required component (Figure 9).** The runs of `scripts/run_cartpole.sh` change one thing at a time and keep everything else, including the selection of the kept network.

   - **No target network** (`--no-target`; the online network computes its own targets; 3 seeds). The values diverge (Figure 10): the visited-state value averages 1.37, 1.51 and 1.69 million over the last 100 episodes, against the bound of 20, and the held-out value 0.87 to 1.22 million. All three final networks end every roll-out after 9.3 to 9.4 steps, the shortest possible episode, and the kept networks score 129.5, 9.4 and 9.4. The same code without a target network does recover Q^* on the chain (C1), whose one-hot states share nothing; on CartPole every update also moves the values of similar states, including the ones it bootstraps from, and without a frozen copy this feedback runs away (function approximation, bootstrapping and off-policy data together, the "deadly triad").
   - **No replay** (`--buffer-size 64`: every update uses the 64 most recent transitions; 3 seeds). On this small problem learning still succeeds: the kept networks score 500 in all 500 roll-outs for all three seeds, and the final networks 89.9, 145.5 and 103.3 (main: 88.6 to 173.4). The value estimate is less accurate, 16.9, 16.9 and 18.5 over the last 100 episodes instead of 19.7 to 20.4, because consecutive, strongly correlated transitions are fitted one after the other. Replay is the component this task needs least; the target network is the one it cannot do without.
   - **DQN target instead of Double DQN** (`--no-double`; 3 seeds). The kept networks score 500 in all 500 roll-outs for all three seeds and the final networks 155.8, 96.6 and 500.0, so on CartPole the choice of target does not decide success. The difference is in the values, in the direction Double DQN is meant to correct, but small: over the last 100 episodes the visited-state value averages 20.00 with the DQN target and 19.92 with Double DQN, and the largest value of an episode 20.99 against 20.85; the spread between seeds (19.72 to 20.37 for Double DQN) is larger than the difference. With only two actions of almost equal value the bias of the maximum is small: for two estimates with equal means and independent noise of standard deviation σ it is $\sigma/\sqrt{\pi} \approx 0.56\sigma$, so a difference of 0.08 corresponds to noise of about 0.15 (an estimate, not a measurement). The overestimation test C3 applied to every kept network gives

+0.67 and +0.47 for the late-kept Double DQN networks (seeds 0 and 3) and +0.35 for the late-kept DQN network (seed 2); networks kept early underestimate by up to 13.9 because their values had not converged, so this test cannot separate the two targets either.

- **Time limit treated as terminal**: test 3.

9. **Control runs.** Two runs check that the update moves the policy in the direction the reward asks for. With **learning rate 0** nothing is learned: the value stays at its initial level ($\max_a Q$ of the first state 0.028 at the start, 0.004 after 300 episodes), and the greedy policy is that of the untrained network (129.5 steps on average over 500 roll-outs). With the **reward negated** (-1 per step) the best the agent can do is to end each episode as fast as possible. It learns exactly that: the final network ends all 500 roll-outs after 8.8 steps on average, every one by letting the pole fall (no run had an episode shorter than 8 steps), and the value of the first state goes to -7.73, close to $-(1-0.95^T)/(1-0.95) = -7.3$ for $T = 8.8$. (The kept network of this run, 129.5, is meaningless: the selection maximises the true CartPole reward, which this run minimises.)

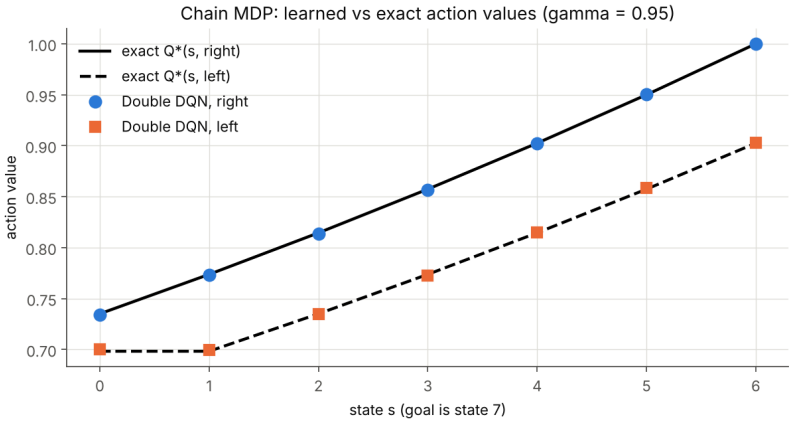


Figure 7: C1: learned (markers) and exact (lines) action values on the goal chain, Double DQN, seed 0.

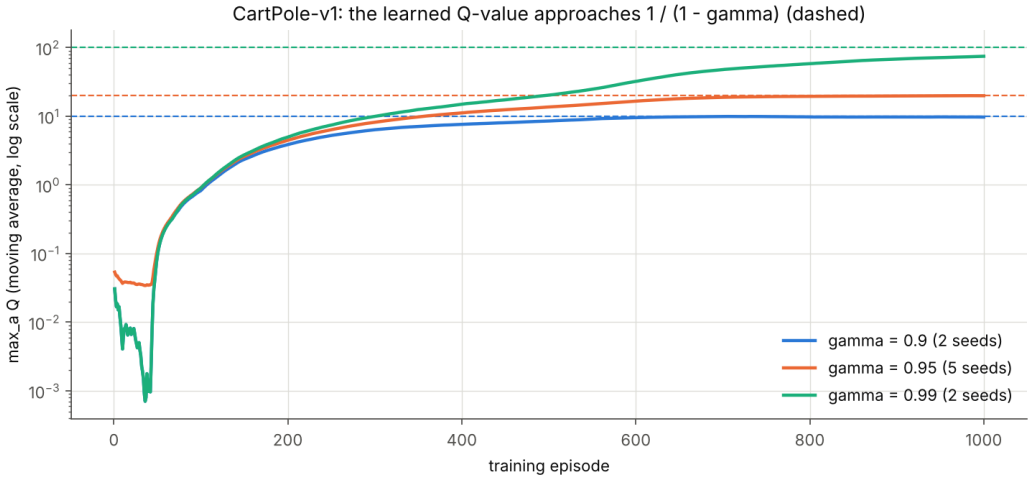


Figure 8: C2: value of the visited states (100-episode moving average, log scale) for discount factors 0.9, 0.95 and 0.99; dashed lines at $1/(1-\gamma)$.

With three more months (part a)

- Repeat every configuration with 20 or more seeds and report confidence intervals for the probability that the kept network solves the task, for the episode at which it is found, and for the size of the

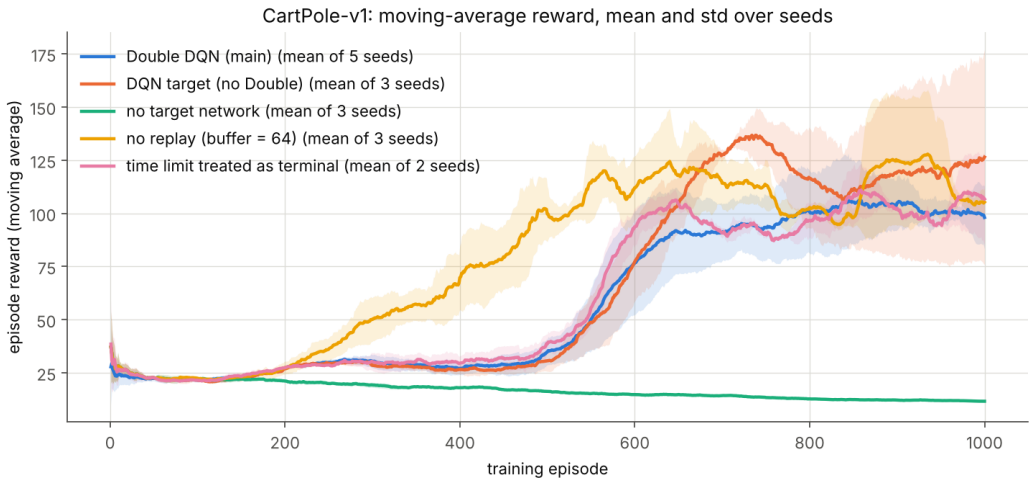


Figure 9: C7: 100-episode moving-average training reward, mean and standard deviation over seeds, for the main configuration and the ablations.

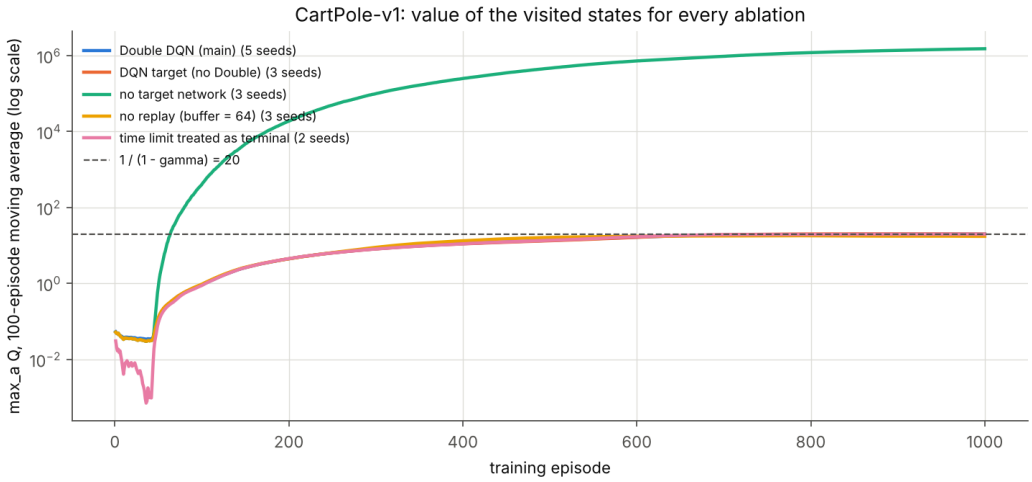


Figure 10: C7: value of the visited states (100-episode moving average, mean over seeds, log scale) for the main configuration and the ablations. Without a target network the values diverge.

collapse after it.

- Measure the action gap $Q(s, a_1) - Q(s, a_2)$ along training and test directly whether the collapses start when the gap falls below the noise of the updates, as argued above. Then test the remedies that act on it: prioritised replay of rare failure transitions, a replay buffer that keeps a fixed share of old episodes, a larger final ϵ, a smaller final learning rate, and the advantage-learning operator that widens the gap.
- Compare the stopping or selection rule with the alternatives (fixed budget, stopping when the moving average reaches 475, selection on 100 instead of 20 evaluation episodes) by their false-positive rate on 500 fresh episodes.
- Test the learned policy beyond its training distribution: random initial states far from upright, pushes during the episode, changed pole length and mass, and the time limit raised to 5000 steps.
- Check $\gamma = 0.95$ against $\gamma = 0.99$ at equal compute: the longer horizon should remove the cart-position failures described in the edge cases.

Part (b): MsPacman-v5

Setup

`ALE/MsPacman-v5` with its defaults: frame skip 4, sticky actions with probability 0.25 (the previous action is repeated regardless of the agent's choice), 9 actions, at most 108,000 frames per game, 3 lives. Every frame goes through the assignment's `preprocess_observation()` unchanged (88 x 80, int8). The state is the stack of the last 4 preprocessed frames, which shows the direction in which Ms. Pac-Man and the ghosts move; after a reset the stack holds four copies of the first frame. The network divides the input by 128.

The network is the original DQN network of Mnih et al. (2013): convolution 8x8 stride 4 (16 filters), convolution 4x4 stride 2 (32), a 256-unit layer and 9 outputs. The larger Mnih et al. (2015) network is implemented as well (`--arch nature`), but on this CPU machine one of its updates took 2.2 times as long (18.8 against 8.6 ms for a minibatch of 32 with 2 threads), so the small network gets about twice as many updates in the same time.

$\gamma = 0.99$ as prescribed. Adam (learning rate 10^{-4} decaying linearly towards 5×10^{-5} over 3 million steps, $\epsilon_{\text{Adam}} = 1.5 \times 10^{-4}$), Huber loss, gradient-norm clipping at 10, minibatch 32, one update every 4 agent steps, target network copied every 1000 updates (4000 steps), replay buffer of 500,000 frames, updates start after 20,000 random steps. ϵ decays from 1 to 0.1 over the next 250,000 steps and then to 0.01 at step 1.5 million. Rewards are clipped to $[-1, 1]$ inside the Bellman target only, and a lost life ends the bootstrap (but not the game); both are edge cases below. All reported scores are raw game scores of complete games.

Results

Learning. Seed 0 trained for 0.92 hours before the machine restart (the reported network is its checkpoint at the end of that time): 1,100 games, 670,642 agent steps (2.7 million frames) and 162,660 updates. Its 100-game moving-average score rose from 277 over the first 100 games (a random policy scores 245) to 1107 over the last 100 (highest moving average 1147, at game 1,084), and the greedy evaluation games played every 100 training games rose from 60 at game 100 to 1103 at game 1,100, the highest of the eleven evaluations (Figure 14). Games also became longer, 514 agent steps on average over the first 100 games and 734 over the last 100.

Seed 1, which trained for 0.25 hours on a single thread, played 300 games (158,928 steps) and its moving average rose from 263 to 512 (Figure 15); after the same 300 games seed 0's moving average was 473.

The handout expects 4,000 to 6,000 games; seed 0 played 1,100, so these runs show the first part of the learning curve rather than its end: over its last 300 games the moving-average score went from 870 to 1107 and the moving-average value of the visited states from 5.19 to 6.75. For scale, the DQN of Mnih et al. (2015) reports 2,311 points on Ms. Pac-Man after 200 million frames under a different evaluation protocol (no sticky actions, $\epsilon = 0.05$, up to 30 no-op starts); seed 0 saw 2.7 million frames.

(i) Maximum Q-values. Measured in clipped-reward units, the value of the visited states rose from 0 to 6.75 (mean over the last 100 games) and the average over the 1000 held-out random-policy states to 6.32; the largest value in a game averaged 13.75. With $\gamma = 0.99$ a value of 6.8 means about that many discounted reward events ahead. 300 games before the end the held-out value was 4.66, so it was still rising when training stopped.

(ii) Scores. The moving average is described above. Single games spread widely: over the last 100 training games they range from 340 to 4160, because of the sticky actions, the 7.1 % of random actions that remain at the end, and the three lives, so the moving average and the greedy evaluation games are the quantities to read.

(iii) Roll-outs. The final network of seed 0 scores 968.8 on average over 500 greedy games, with standard deviation 362.0 (minimum 460, median 870, maximum 3100), 4.0 times the random policy; 100.0 % of its games score more than the best of the 100 random games (410). The network of seed 1, after only 300 games, scores 141.2 (standard deviation 10.9): 494 of its 500 greedy games end with exactly 140 points, less than the random policy. Acting greedily, this young network repeats one losing route whatever the sticky actions do, although the same network averaged 512 in training, where half of its actions ($\epsilon = 0.50$) were random; this is the stuck-greedy-policy edge case below, and seed 0 shows that it disappears with more training (its greedy evaluations were 60 and 170 at games 100 and 200 and above 850 from game 700 on, except one evaluation of 471 at game 800).

Continued training. After the first restart, training of seed 0 was resumed from its checkpoint at game 1,100 (`scripts/run_mspacman_continue.sh`; the replay buffer, which is not checkpointed, was refilled for 50,000 steps). Further restarts stopped it again; its checkpoint at game 1,500 (945,654 agent steps, 1.34 hours of training in total; `results/mspacman/cont_seed0`) scores 1074.9 on average over 500 greedy games (standard deviation 385.8, median 1040, maximum 3150), against 968.8 for the network of game 1,100: 106 points more, about 4.5 standard errors of the difference (treating the two sets of games as independent). Learning was still continuing when the machine stopped it. Seed 1 was continued in the same way to game 700.

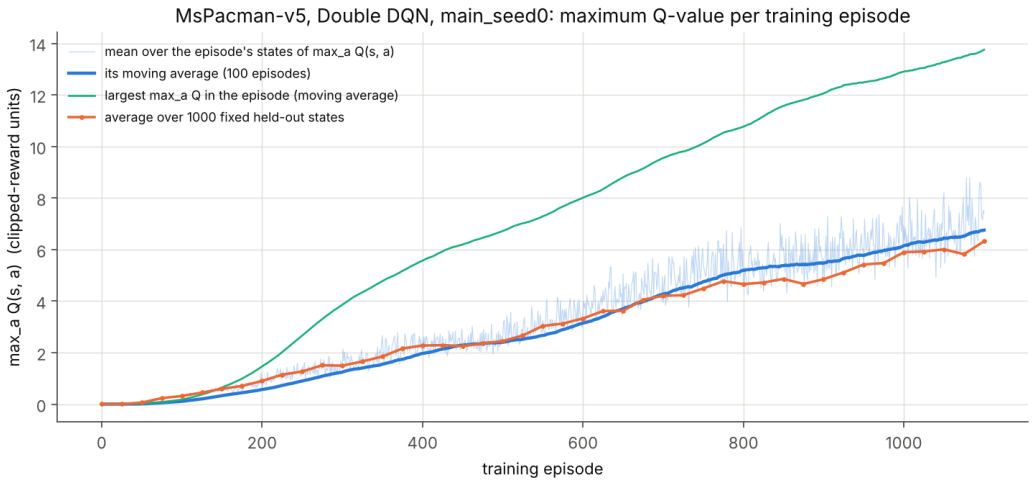


Figure 11: (b)(i) MsPacman-v5: maximum Q-value per training episode (clipped-reward units).

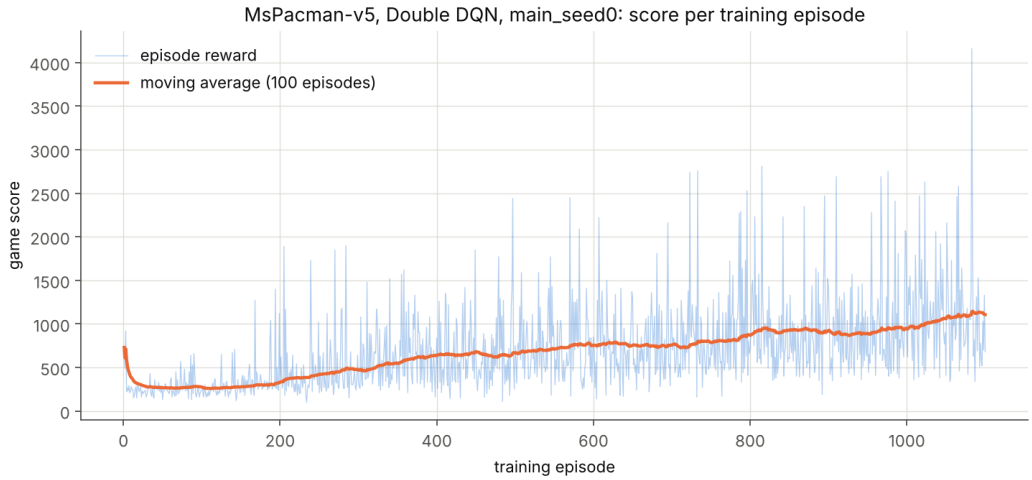


Figure 12: (b)(ii) MsPacman-v5: score per training episode and its 100-episode moving average.

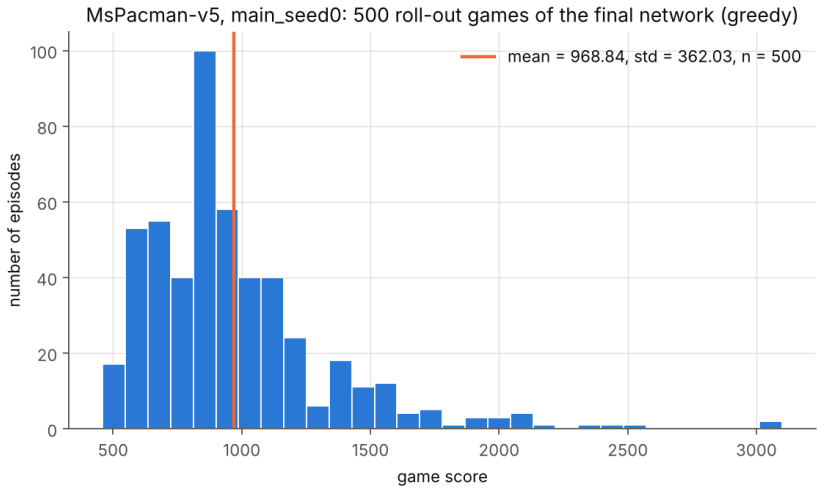


Figure 13: (b)(iii) MsPacman-v5: 500 greedy roll-out games of the final network.

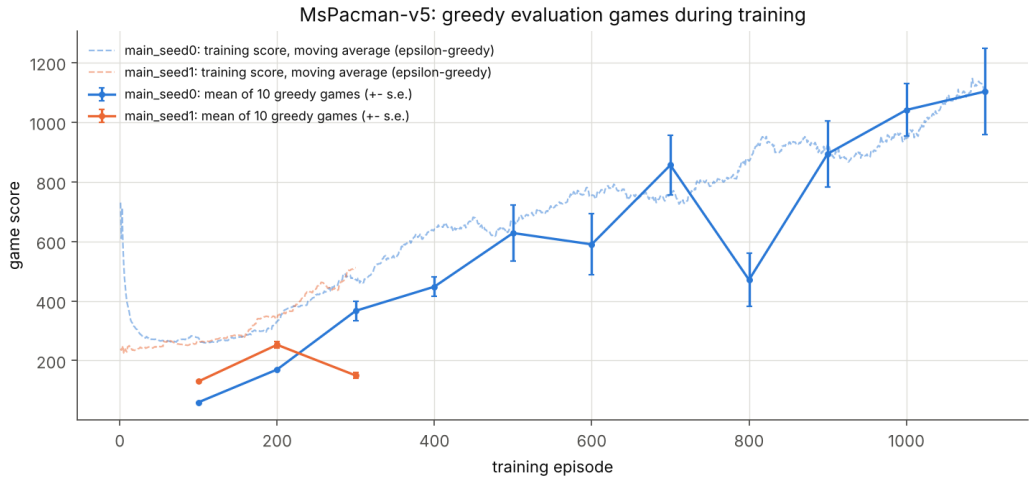


Figure 14: MsPacman-v5: greedy evaluation games (10 every 100 training episodes) next to the training score.

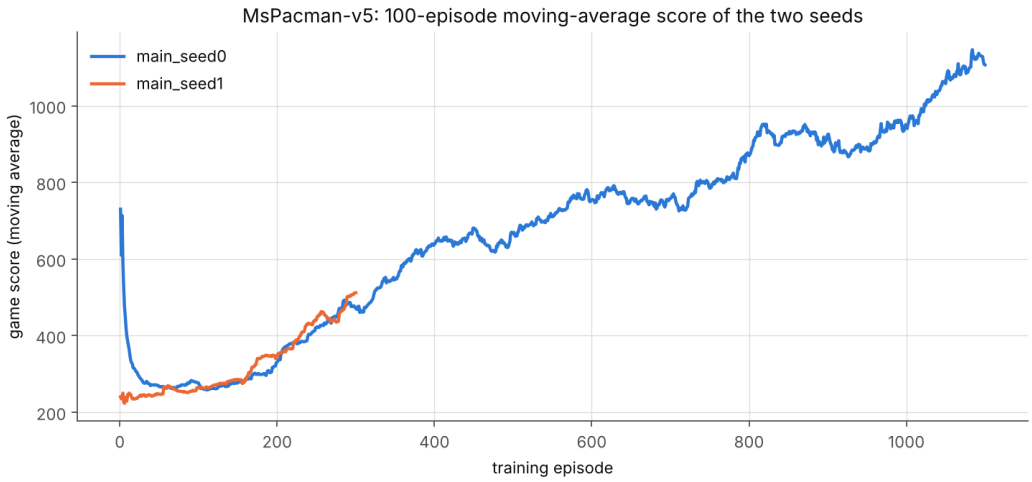


Figure 15: MsPacman-v5: 100-game moving-average score of the two seeds (seed 1 trained for a shorter time on one thread).

Edge cases (part b)

- **The given preprocessing function computes in unsigned integers.** The observation is `uint8`, so `img.sum(axis=2)` is `uint64` and `img // 3 - 128` wraps around to huge numbers for every pixel darker than 128 (most of the screen). The final `astype(np.int8)` keeps the lowest 8 bits, which equal the intended signed value, so the function is correct by a coincidence of modular arithmetic. It is used unchanged, and a unit test compares it with a signed computation on a random frame (identical). In a real game its output lies in $[-128, 27]$ (the brightest colour sum is 466), so the network's input $x/128$ lies in $[-1, 0.21]$.
- **The contrast trick must hit Ms. Pac-Man and nothing else.** Pixels whose colour channels sum to 448 are set to the darkest value. Over 449 frames of a random game (test P1, Figure 16) 7 to 17 pixels of the cropped frame had this sum in every frame (mean 10.7), and no pixel of any other colour had it, so exactly Ms. Pac-Man is darkened and she is never lost from the input.
- **One frame shows no motion, and states must not mix two games.** The state stacks the last 4 frames, which shows where Ms. Pac-Man and the ghosts are heading; after a reset the stack holds 4 copies of the first frame. The replay buffer stores every frame once (500,000 frames, 3.5 GB) and rebuilds the stacks at sampling time; a frame that belongs to the previous game is replaced by the first frame of the current one, the same rule as in the acting loop, and once the buffer is full the 4 oldest slots, the next to be overwritten, are never sampled because their stacks would reach into the newest frames. A unit test replays 60 random-length episodes through buffers of 37, 50 and 1000 slots (forcing wrap-around) and checks that every transition the buffer can return equals the one the acting loop saw.
- **Lives.** A game has 3 lives. A lost life ends the bootstrap (the target is the clipped reward alone), but not the game: the frame stack continues and the reported score is the full game. The crop removes the bottom of the screen where the remaining lives are drawn (checked: the bottom rows of the preprocessed frame do not change at any of the three life losses of a game), so the network cannot see how many lives are left; treating each life as an episode for the target is consistent with that.
- **Sticky actions.** With probability 0.25 the emulator repeats the previous action whatever the agent chose. The stored action is the one the agent chose, which is the action whose value the update is about; the repetition is part of the transition noise. Test P7 removes it to make the game deterministic.

- **Reward scale.** A pellet is worth 10, a power pellet 50, ghosts 200 to 1600 and fruit 100 to 5000. The target uses rewards clipped to $[-1, 1]$, so Q is the discounted number of rewarding steps ahead (at most $1/(1-\gamma) = 100$) and the maximum-Q plot is in these units. The price is that a ghost is worth as much as a pellet; test P9 measures what the agent eats.
- **The time limit.** A game is truncated after 108,000 frames (27,000 agent steps). The truncation is bootstrapped like CartPole's, but it never happened: the longest training game had 1,381 steps.
- **A greedy policy can get stuck.** The first two greedy evaluations (episodes 100 and 200) gave the same score in all 10 games (60 and 170, standard deviation 0) although the sticky actions make every game different. The early greedy policy therefore repeats the same path whatever the noise, most likely pushing into a wall until the ghosts end the game (an interpretation, not checked frame by frame). ϵ-greedy exploration prevents this during training, and the greedy evaluations show when the policy has outgrown it.
- **Compute and interruptions.** The runs had fixed wall-clock budgets, and a restart of the cloud machine stopped both (seed 0 after 1,100 games, seed 1 after 300; the reported networks are the checkpoints written every 50 games). All schedules were shortened to fit the budget (learning starts after 20,000 steps instead of 50,000; ϵ reaches 0.01 at 1.5 million steps instead of 2.5 million), and the replay buffer, which is not part of the checkpoint (3.5 GB), is refilled for 50,000 steps when a run resumes.

Tests that validate correctness (part b)

`python -m dqn.validate_mspacman --run results/mspacman/main_seed0` runs P1 to P9 on the trained network and writes `results/mspacman/validation/validation.md`.

1. **P1: preprocessing (Figure 16).** The raw frame, the preprocessed frame and the four frames of a state. The function is used unchanged; its unsigned arithmetic gives the signed result (unit test), the colour sum 448 marks Ms. Pac-Man and nothing else (7 to 17 pixels in each of 449 frames, no other colour with that sum), and the output range in a game is $[-128, 27]$.
2. **P2: random policy.** 100 games with uniformly random actions score 245.1 on average (standard deviation 50.4, minimum 140, maximum 410) and last 495 agent steps. This is the floor every learning curve must rise above.
3. **P3: learning without exploration noise (Figure 17).** Every saved network played 20 greedy games (sticky actions on, new seeds), so the exploration noise of the training curve is absent. Mean score by training game of the snapshot: 0: 332, 100: 60, 250: 335, 500: 462, 1,000: 1000, 1,100: 891. The untrained network scores 332 and the last one 891 (standard error 74; the 500 roll-out games of the same network average 968.8). The network of game 100 scores 60 in all 20 games: the stuck greedy policy of the edge cases, which more training removes.
4. **P4: does the network predict what it is trained to predict? (Figure 18)** The network is trained to predict the discounted sum of clipped rewards up to the next lost life. Along 20 greedy games (15,142 steps) the predicted $Q(s_t, a_t)$ averages 6.82 and the realised return of the same policy 6.55: the network overestimates by 0.27 on average. Prediction and outcome are weakly correlated (correlation 0.25); sorted into 20 bins of the prediction, the realised return goes from 13.12 in the lowest bin (prediction 1.85) to 16.07 in the highest (prediction 13.20). The overestimation on average hides two regimes: the highest predictions are close to their outcome, while the lowest are far below it (1.85 predicted, 13.12 realised). The likely candidates for these are the frozen starts of a game and of each life, where the network does not track the value (test P7); this was not checked state by state.
5. **P5: value around game events, the non-standard plot (Figure 19).** For every event in 20 greedy games the trace of $\max_a Q$ is aligned on the event and averaged: around a lost life (n = 40) the average $\max_a Q$ is 7.36 sixty steps before, 6.57 ten steps before, 5.86 at the event and 4.04 ten steps after; around a power pellet (n = 21) the average $\max_a Q$ is 7.92 sixty steps before, 7.81 ten

steps before, 7.18 at the event and 7.01 ten steps after; around an eaten ghost (n = 17) the average $\max_a Q$ is 7.14 sixty steps before, 6.63 ten steps before, 6.63 at the event and 6.55 ten steps after. Before a lost life the value falls as the ghosts close in and drops at the event; it stays low during the death animation, when no reward can be collected. A power pellet does not raise the value (7.81 ten steps before, 7.01 ten steps after), which is consistent with the clipped reward: a ghost is worth no more than a pellet to this agent (test P9).

6. **P6: the first 10 steps of a greedy game.** The table follows the final network from the start of a game. During these steps the game is still in its frozen intro (test P7), so no action has any effect, the reward is 0, and the Bellman target is $0.99 \max_a Q(s')$.

t	lives	Ms. Pac-Man (row, col)	max Q	argmax	max Q - min Q	action	reward	y = clip(r) + 0.99 max Q(s')
0	3	(103, 80)	1.970	UPLEFT	0.0678	UPLEFT	0	1.924
1	3	(103, 79)	1.944	UPLEFT	0.0789	UPLEFT	0	2.012
2	3	(103, 80)	2.032	UPLEFT	0.0848	UPLEFT	0	1.772
3	3	(103, 80)	1.790	UPLEFT	0.0744	UPLEFT	0	1.818
4	3	(103, 80)	1.836	UPLEFT	0.0747	UPLEFT	0	1.921
5	3	(103, 79)	1.940	UPLEFT	0.0823	UPLEFT	0	1.898
6	3	(103, 80)	1.917	UPLEFT	0.0771	UPLEFT	0	1.803
7	3	(103, 80)	1.821	UPLEFT	0.0781	UPLEFT	0	1.829
8	3	(103, 80)	1.847	UPLEFT	0.0789	UPLEFT	0	1.976
9	3	(103, 79)	1.996	UPLEFT	0.0792	UPLEFT	0	1.922

7. **P7: determinism and the frozen start.** With sticky actions switched off, the emulator and the greedy policy are both deterministic, so games started with different seeds must be identical: the five games scored 870, 870, 870, 870, 870 (identical, also in length). With the default sticky actions the same five seeds give 600, 710, 1140, 780, 1040. At each of the first 120 steps of a game the emulator was cloned and all 9 actions were applied: in the first 66 steps every action leads to exactly the same emulator state (66 frozen steps in all), the intro of the game. Where actions cannot matter the network's action gap $\max_a Q - \min_a Q$ averages 0.081, against 0.375 afterwards. During the intro nothing can happen except that the first reward comes one step closer, so $\max_a Q$ should grow by the factor $1/\gamma$ per step, i.e. $\log \max_a Q$ with slope $-\log 0.99 = 0.0101$; the fitted slope over the 66 frozen steps is 0.0002 ($\max_a Q$ from 1.970 to 2.159). The network does not reproduce it (2 % of the predicted slope). A likely reason (not tested): the frames of the intro differ only by small animations, so the network gives them almost the same value, and because no action matters there, this error never changes a decision.
8. **P8: where Ms. Pac-Man goes, a second non-standard plot (Figure 20).** Ms. Pac-Man's position was read from her colour in the raw frames of 10 greedy games per network. The network after episode 250 visited 7.3 % of the 4 x 4-pixel cells of the screen (5,912 positions); the final network visited 24.5 % of the 4 x 4-pixel cells of the screen (8,430 positions), more than three times the area, mostly new corridors in the upper half of the maze (Figure 20).
9. **P9: what the agent eats.** Over 20 greedy games the final network scores 985 per game, of which 24.0 % comes from rewards of 100 points or more (ghosts and fruit); it eats 1.05 power pellets and 0.85 ghosts per game. The random policy scores 272, 3.7 % from such rewards, with 0.10 power pellets and 0.05 ghosts per game. Reward values seen (value: count): 10: 1392, 50: 21, 100: 5, 200: 10, 210: 3, 400: 4.

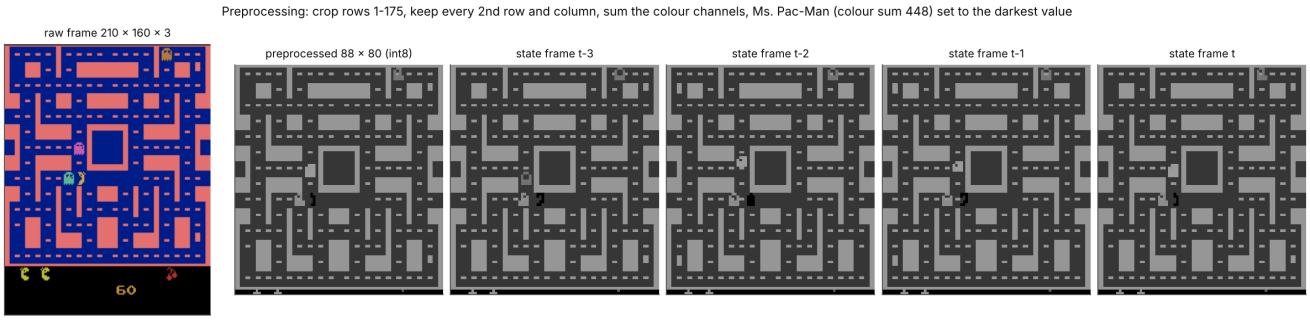


Figure 16: P1: raw frame, preprocessed frame and the four preprocessed frames that form one state.

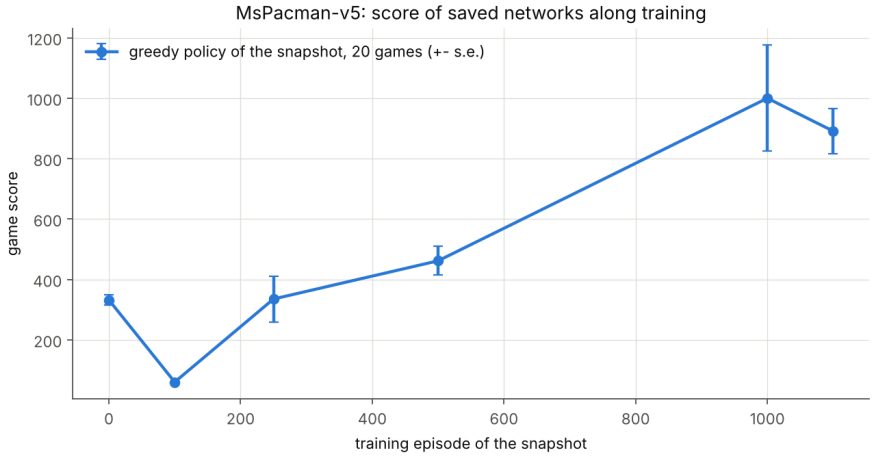


Figure 17: P3: score of the saved networks along training, 20 greedy games each.

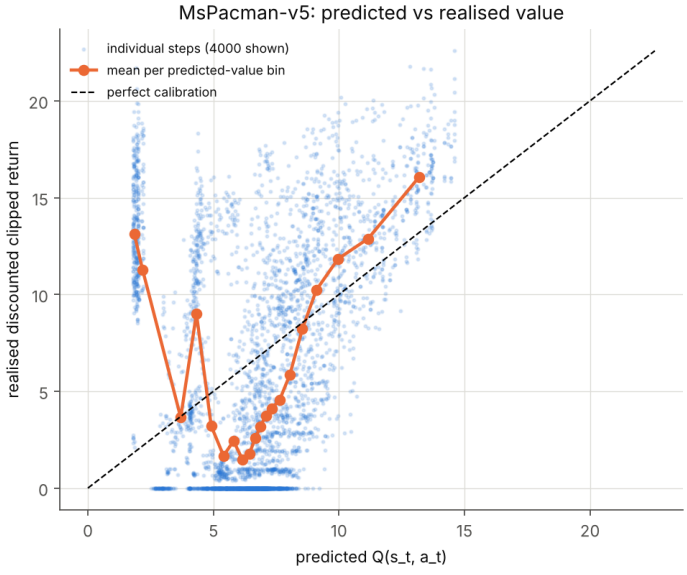


Figure 18: P4: predicted $Q(s_t, a_t)$ against the realised discounted clipped return of the same greedy policy (cut at every lost life, as in the training target).

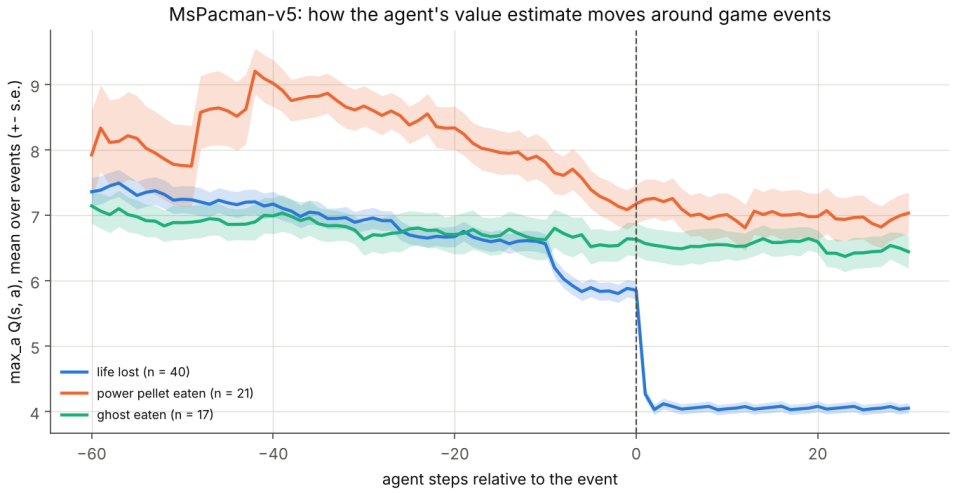


Figure 19: P5 (non-standard plot): average of $\max_a Q$ in the 60 steps before and 30 steps after a lost life, a power pellet and an eaten ghost.

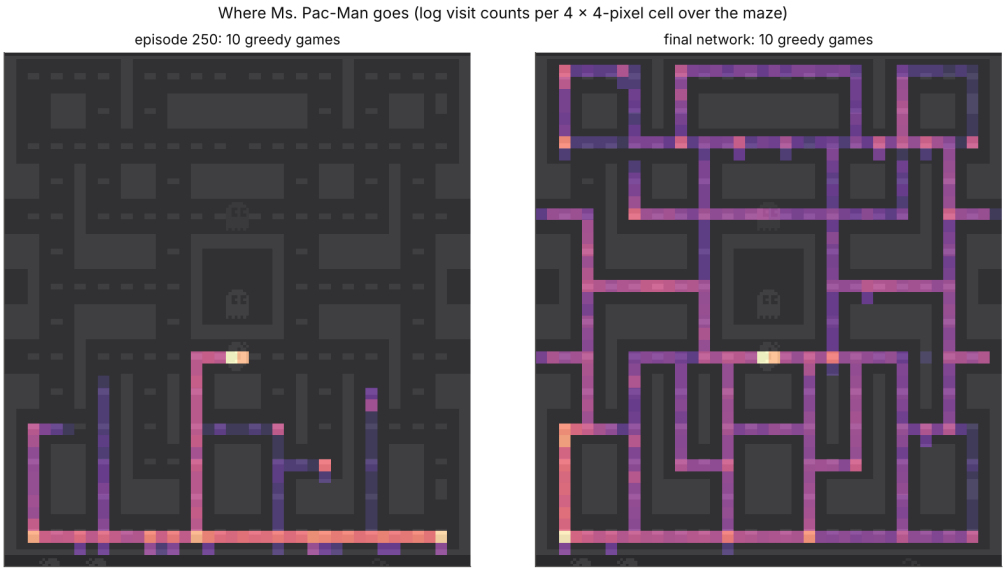


Figure 20: P8 (non-standard plot): where Ms. Pac-Man goes, early in training and with the final network.

With three more months (part b)

- Train for the full 4000 to 6000 episodes (10 to 50 million frames) with several seeds on a GPU, and report the learning curves with confidence bands; the run here is one seed and a short budget.
- Ablate each design choice on MsPacman at equal compute: reward clipping against a log or Pop-Art normalised reward (clipping makes a ghost worth as much as a pellet), life loss as terminal or not, Double against plain DQN, the 2013 against the 2015 network, the frame stack against a max-pooled pair of frames.
- Add the standard improvements one at a time (prioritised replay, dueling network, n-step returns, noisy networks) and measure their effect on the same seeds.
- Extend the calibration test: compare Q with Monte Carlo returns state by state with many roll-outs from cloned emulator states, so that the overestimation can be measured without the noise of a single trajectory.
- Probe what the network has learned with saliency maps and with controlled states (ghost placed next to Ms. Pac-Man, power pellet just eaten), and test robustness to sticky-action probability, frame skip and the other mazes of the game.